

FIAP – FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA.
CLOUD COMPUTING

EDMÁRIO SANTOS
PEDRO GONÇALVES
RONALDO ATTAMAH
KAUÊ LIMA

SEGURANÇA INTEGRADA AO CICLO DE DESENVOLVIMENTO

laboratório DevSecOps com SonarQube, Trivy, KICS e Nuclei

Prof. Fábio Pires

São Paulo
2026

RESUMO

Este trabalho documenta o CP01-DevSecOps, laboratório didático que compara estados vulnerável e corrigido de uma aplicação Python, suas dependências e um manifesto Kubernetes. São estudadas quatro categorias: SAST com SonarQube Community Build, SCA com Trivy, IaC com KICS e DAST com Nuclei. O Docker Compose fixa as imagens externas por versão e digest; os alvos são construídos localmente e isolados em rede interna. O GitHub Actions executa Trivy e KICS e reprova o job diante de achados HIGH ou CRITICAL. Foram identificadas três vulnerabilidades MEDIUM no Jinja2 3.1.4 e dois achados HIGH de privilégios no manifesto. Após as correções, o gate passou sem bloqueantes, mantendo os avisos menores. O laboratório Trivy/KICS foi reproduzido por Cauê no Windows e por Ronaldo no macOS, com assistência declarada do Codex e supervisão dos integrantes, produzindo contagens coincidentes. Nuclei e SonarQube possuem evidências complementares no Mac. As configurações utilizadas examinaram superfícies diferentes; o resultado verde expressa conformidade com a política adotada e não ausência total de risco. O roteiro da turma foi planejado para até 12 minutos após os downloads, mas o ensaio explicado ainda não está comprovado nas evidências consultadas.

Palavras-chave: DevSecOps; análise estática; análise de composição de software; infraestrutura como código; integração contínua.

LISTA DE ILUSTRAÇÕES

Quadro 1 – Requisitos funcionais do laboratório	8
Quadro 2 – Requisitos não funcionais do laboratório	9
Figura 1 – Posicionamento das categorias de análise no ciclo de desenvolvimento	10
Figura 2 – Arquitetura da solução CP01-DevSecOps	11
Quadro 3 – Stack tecnológica e versões fixadas	12
Quadro 4 – Comparativo das ferramentas utilizadas	16
Quadro 5 – Endpoints da aplicação-alvo	17
Figura 3 – Fluxo do gate de segurança no GitHub Actions	18
Código-fonte 1 – Clonagem, conferência do ambiente e preparo dos downloads	19
Quadro 6 – Diferenças entre os estados vulnerável e corrigido	20
Quadro 7 – Variáveis de ambiente e argumentos de build	21
Quadro 8 – Níveis do plano de testes	23
Quadro 9 – Casos de teste e resultados esperados	23
Quadro 10 – Resultados consolidados por ferramenta	24
Quadro 11 – Triagem dos três achados principais	25
Quadro 12 – Comparação das validações independentes	26
Quadro 13 – Ocorrências registradas e orientações preventivas	27
Código-fonte 2 – Roteiro principal do laboratório (Trivy + KICS)	34
Código-fonte 3 – Automação do gate, DAST, SAST e encerramento	35
Código-fonte 4 – Workflow completo .github/workflows/security.yml	36
Código-fonte 5 – Trecho do docker-compose.yml: endurecimento dos alvos e serviço Trivy	37
Código-fonte 6 – app/Dockerfile	37
Código-fonte 7 – Trecho do securityContext no manifesto corrigido	38
Código-fonte 8 – nuclei/debug-exposure.yaml	38

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Delivery
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
GHSA	GitHub Security Advisory
HTTP	Hypertext Transfer Protocol
IaC	Infrastructure as Code
JSON	JavaScript Object Notation
KICS	Keeping Infrastructure as Code Secure
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
RF	Requisito Funcional
RN	Regra de Negócio
RNF	Requisito Não Funcional
SARIF	Static Analysis Results Interchange Format
SAST	Static Application Security Testing
SBOM	Software Bill of Materials
SCA	Software Composition Analysis
SSDF	Secure Software Development Framework
TLS	Transport Layer Security
YAML	YAML Ain't Markup Language

SUMÁRIO

1 INTRODUÇÃO	6
1.1 CONTEXTUALIZAÇÃO	6
1.2 PROBLEMATIZAÇÃO	6
1.3 OBJETIVOS	6
1.3.1 Objetivo geral	6
1.3.2 Objetivos específicos.....	6
1.4 JUSTIFICATIVA	7
1.5 METODOLOGIA E ESCOPO	7
2 REQUISITOS DO SISTEMA	8
2.1 ATORES E PREMISSAS	8
2.2 REQUISITOS FUNCIONAIS	8
2.3 REQUISITOS NÃO FUNCIONAIS.....	9
2.4 REGRAS DA POLÍTICA DE SEGURANÇA.....	9
3 ARQUITETURA E TECNOLOGIAS	10
3.1 FUNDAMENTAÇÃO: CATEGORIAS DE ANÁLISE.....	10
3.2 VISÃO GERAL DA ARQUITETURA.....	11
3.3 STACK TECNOLÓGICA	12
3.4 FERRAMENTAS DE SEGURANÇA	12
3.4.1 SonarQube Community Build (SAST).....	12
3.4.2 Trivy (SCA).....	13
3.4.3 KICS (IaC).....	14
3.4.4 Nuclei (DAST)	15
3.4.5 Quadro comparativo	16
3.5 APLICAÇÃO-ALVO E API	17
3.6 PERSISTÊNCIA E ARTEFATOS DE DADOS	17
3.7 PIPELINE DE INTEGRAÇÃO CONTÍNUA	17
4 IMPLEMENTAÇÃO E MANIFESTOS	19
4.1 PRÉ-REQUISITOS DO AMBIENTE	19
4.2 OBTENÇÃO DO PROJETO E PREPARAÇÃO	19
4.3 ORQUESTRAÇÃO COM DOCKER COMPOSE.....	20
4.4 DOCKERFILE DA APLICAÇÃO	20
4.5 MANIFESTOS KUBERNETES VULNERÁVEL E CORRIGIDO	20
4.6 TEMPLATE NUCLEI	21
4.7 VARIÁVEIS DE AMBIENTE E CONFIGURAÇÃO	21

4.8 GUIA DE EXECUÇÃO.....	21
4.9 WORKFLOW DO GITHUB ACTIONS	22
5 TESTES E VALIDAÇÃO.....	23
5.1 PLANO DE TESTES	23
5.2 CENÁRIOS E CASOS DE TESTE.....	23
5.3 RESULTADOS OBTIDOS	24
5.4 ANÁLISE DOS ACHADOS PRINCIPAIS	24
5.5 VALIDAÇÃO EM AMBIENTES INDEPENDENTES	26
5.6 DIFICULDADES ENCONTRADAS E TRATAMENTO	27
6 CONSIDERAÇÕES FINAIS	28
6.1 GANHOS ALCANÇADOS	28
6.2 LIMITAÇÕES	28
6.3 TRABALHOS FUTUROS	28
REFERÊNCIAS	29
APÊNDICE A – DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL	33
APÊNDICE B – COMANDOS DE EXECUÇÃO E WORKFLOW	34
APÊNDICE C – TRECHOS ILUSTRATIVOS DE CONFIGURAÇÃO	37

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO

Humble e Farley (2010) descrevem o pipeline como sequência automatizada de compilação, testes e empacotamento. Kim et al. (2016) defendem integrar segurança ao trabalho diário. Neste contexto, DevSecOps incorpora controles ao mesmo fluxo usado para desenvolver e entregar software.

A abordagem shift-left antecipa a identificação de problemas. O SSDF recomenda verificação automatizada e gestão de componentes de terceiros (Souppaya; Scarfone; Dodson, 2022). O laboratório aplica esses princípios ao código próprio, às dependências e aos manifestos, complementando-os com um teste HTTP em execução.

Essas superfícies exigem técnicas distintas. O trabalho compara o que cada configuração detecta e quais riscos permanecem fora do ensaio.

1.2 PROBLEMATIZAÇÃO

O problema é combinar scanners sem confundir ausência de achados com ausência de risco, falha operacional com bloqueio de segurança ou versões fixadas com uma base de vulnerabilidades imutável.

Como comparar SAST, SCA, IaC e DAST e automatizar um gate Trivy/KICS que distinga aprovação, achados bloqueantes e erro operacional?

1.3 OBJETIVOS

1.3.1 Objetivo geral

Desenvolver e documentar um laboratório DevSecOps reproduzível com as quatro ferramentas atribuídas ao Grupo 2, comparar estados vulnerável e corrigido e aplicar no CI um gate por severidade com Trivy e KICS. As licenças dos componentes são discriminadas na seção 3.4.

1.3.2 Objetivos específicos

- a) caracterizar as categorias SAST, SCA, IaC e DAST e posicioná-las no ciclo de desenvolvimento;
- b) construir uma aplicação-alvo e *fixtures* de dependências e manifestos Kubernetes nos estados vulnerável e corrigido;

- c) orquestrar SonarQube Community Build, Trivy, KICS e Nuclei com Docker Compose, fixando versões e digests das imagens;
- d) implementar no GitHub Actions um gate que reprove achados HIGH ou CRITICAL e diferencie falhas de segurança de falhas operacionais;
- e) analisar três achados reais com identificação de CWE, triagem de verdadeiro ou falso positivo e correção comprovada;
- f) validar a reprodução do laboratório em pelo menos duas máquinas e sistemas operacionais distintos.

1.4 JUSTIFICATIVA

O laboratório foi planejado para até 12 minutos após os downloads, permitindo observar detecção, correção e reprovação/aprovação do gate. O ensaio explicado com esse limite ainda precisa ser registrado; os tempos automatizados não o substituem. O experimento aborda componentes vulneráveis e configuração insegura, classes presentes no OWASP Top 10 de 2021 (OWASP Foundation, 2021).

A execução local em contêineres evita expor os alvos à internet e dispensa licenças comerciais para este laboratório. A seleção segue o enunciado do Grupo 2; no SonarQube, a licença da plataforma deve ser distinguida daquela dos analisadores incluídos.

1.5 METODOLOGIA E ESCOPO

Compararam-se fixtures vulneráveis e corrigidas com as mesmas versões de scanners. Os relatórios brutos e resumos foram preservados no repositório para auditoria das contagens (Grupo 2, 2026).

Adotam-se A4, Arial 12, entrelinhas 1,5 no corpo e margens de 3 cm (superior/esquerda) e 2 cm (inferior/direita). Referências e elementos especiais usam espaçamento próprio. As edições NBR 14724:2024 e 6023:2025 foram conferidas nas orientações da Ufes (Universidade Federal do Espírito Santo, [s.d.]).

O LAB da turma usa Trivy/KICS; Nuclei/SonarQube são complementares. Não houve cluster, alvo externo ou exploração: os achados demonstram exposição e configuração insegura.

2 REQUISITOS DO SISTEMA

Os requisitos derivam do enunciado e das decisões de projeto. Artefatos apontam sua implementação; a seção 5 distingue testes registrados de controles ainda sem prova negativa.

2.1 ATORES E PREMISSAS

O integrante mantém o projeto e interpreta as quatro ferramentas; o participante executa o LAB Trivy/KICS; o GitHub Actions executa o gate em PR e push para main.

São necessários Git, Docker com Compose e internet no preparo. No Windows, usar contêineres Linux e WSL 2.

2.2 REQUISITOS FUNCIONAIS

Quadro 1 – Requisitos funcionais do laboratório

Código	Descrição	Verificação
RF01	Analisar as dependências declaradas em requirements.txt e listar vulnerabilidades conhecidas com CVE, severidade e versão corrigida.	Relatório Trivy JSON
RF02	Analisar o manifesto Kubernetes deployment.yaml e reportar configurações inseguras com severidade e CWE.	Relatório KICS JSON/SARIF
RF03	Analisar o código-fonte próprio (app/server.py) em busca de fragilidades e <i>security hotspots</i> .	Exportação SonarQube
RF04	Testar dinamicamente a aplicação em execução com um template HTTP próprio.	Relatório Nuclei JSONL
RF05	Disponibilizar os estados vulnerável e corrigido para comparação direta com os mesmos scanners.	Diretórios fixtures/
RF06	Bloquear o pipeline quando houver ao menos um achado HIGH ou CRITICAL no Trivy ou no KICS.	Run vermelho 36167909434
RF07	Aprovar o pipeline quando não houver achados acima do limiar, mantendo visíveis os avisos menores.	Run verde 36167707788
RF08	Tratar ausência ou invalidez de relatório como erro operacional (código 2), nunca como aprovação.	scripts/test_gate.py
RF09	Exportar resultados em JSON, SARIF e SBOM CycloneDX e preservá-los mesmo quando o job falhar.	Artifacts do workflow
RF10	Gerar comprovante pessoal de execução que rejeite relatórios com mais de duas horas.	scripts/comprovante.py
RF11	Permitir a execução manual do workflow com escolha do cenário (vulnerável ou corrigido).	workflow_dispatch

Fonte: Elaborado pelos autores (2026).

2.3 REQUISITOS NÃO FUNCIONAIS

Quadro 2 – Requisitos não funcionais do laboratório

Código	Categoria	Descrição
RNF01	Reprodutibilidade	Fixar imagens externas e base Python por versão/digest; construir os alvos locais com tags e fontes versionadas.
RNF02	Portabilidade	O laboratório deve executar em Windows (WSL 2), macOS e Linux sem instalar scanners no sistema hospedeiro.
RNF03	Desempenho	Meta: roteiro explicado em até 12 minutos após downloads; ensaio ainda não comprovado.
RNF04	Isolamento	Os alvos DAST devem operar em rede Docker interna, sem portas publicadas nem saída para a internet.
RNF05	Menor privilégio	Os contêineres da aplicação devem rodar como usuário não root, com sistema de arquivos somente leitura e sem capabilities.
RNF06	Limitação de recursos	Cada alvo deve ter limite de 256 MB de memória, 0,5 CPU e 64 processos; o SonarQube, de 3 GB.
RNF07	Confidencialidade	Credenciais do SonarQube devem ficar apenas no arquivo .env, ignorado pelo Git.
RNF08	Rastreabilidade	Relatórios brutos devem ser versionados, com data da base de vulnerabilidades registrada.
RNF09	Operação offline	Após o preparo, o Trivy deve analisar sem nova consulta à rede (--offline-scan).
RNF10	Transparência	O uso de inteligência artificial deve ser declarado e as contribuições reais atribuídas a seus autores.

Fonte: Elaborado pelos autores (2026).

2.4 REGRAS DA POLÍTICA DE SEGURANÇA

As regras a seguir definem o comportamento do gate e foram implementadas no script `scripts/run_scan.py`.

- a) **RN01 – HIGH/CRITICAL bloqueiam; severidades menores permanecem visíveis.**
- b) **RN02 – Manter a severidade da fonte adotada pelo scanner.**
- c) **RN03 – Não aplicar supressões ou exceções neste laboratório.**
- d) **RN04 – Retornar 0 PASS, 1 FAIL ou 2 ERROR; erro operacional tem prioridade.**
- e) **RN05 – Apagar relatórios anteriores antes de cada análise.**

3 ARQUITETURA E TECNOLOGIAS

3.1 FUNDAMENTAÇÃO: CATEGORIAS DE ANÁLISE

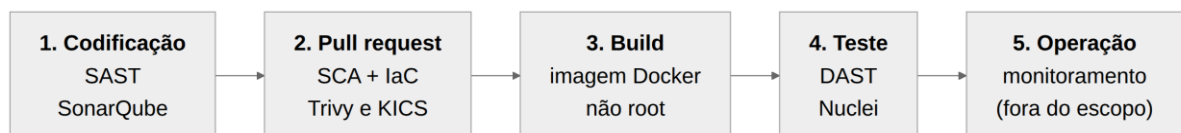
O **SAST** examina o código-fonte sem executá-lo, buscando padrões associados a fragilidades, como uso de algoritmos criptográficos fracos, injeções ou comunicação sem criptografia. Sua força está em apontar a linha exata do problema no código que a própria equipe escreveu; sua limitação é não conhecer o ambiente de execução e, por isso, produzir alertas que exigem triagem contextual.

O **SCA** identifica componentes de terceiros e suas versões e as confronta com bases públicas de vulnerabilidades conhecidas (CVE e *advisories*). Enquanto o SAST pergunta “o nosso código está correto?”, o SCA pergunta “as bibliotecas que usamos têm falhas já publicadas?”. Um subproduto natural do SCA é o **SBOM**, inventário estruturado dos componentes do software; neste projeto é gerado no formato CycloneDX (CycloneDX, [s.d.]). Um SBOM é um inventário, não um certificado de segurança.

A análise de **IaC** aplica regras a arquivos declarativos de infraestrutura – manifestos Kubernetes, Terraform, Dockerfiles – antes que sejam implantados. Configurações como contêineres privilegiados ou escalada de privilégios permitida são detectadas no próprio arquivo, sem a necessidade de um cluster, o que se alinha às recomendações do NIST para segurança de contêineres (Souppaya; Morello; Scarfone, 2017).

O **DAST** observa a aplicação em execução, enviando requisições e analisando respostas. Ele detecta problemas que só se manifestam em tempo de execução, como um endpoint de depuração exposto, mas cobre apenas os caminhos e condições efetivamente testados. A Figura 1 posiciona as quatro categorias ao longo do ciclo de desenvolvimento adotado no laboratório.

Figura 1 – Posicionamento das categorias de análise no ciclo de desenvolvimento



Fonte: Elaborado pelos autores (2026).

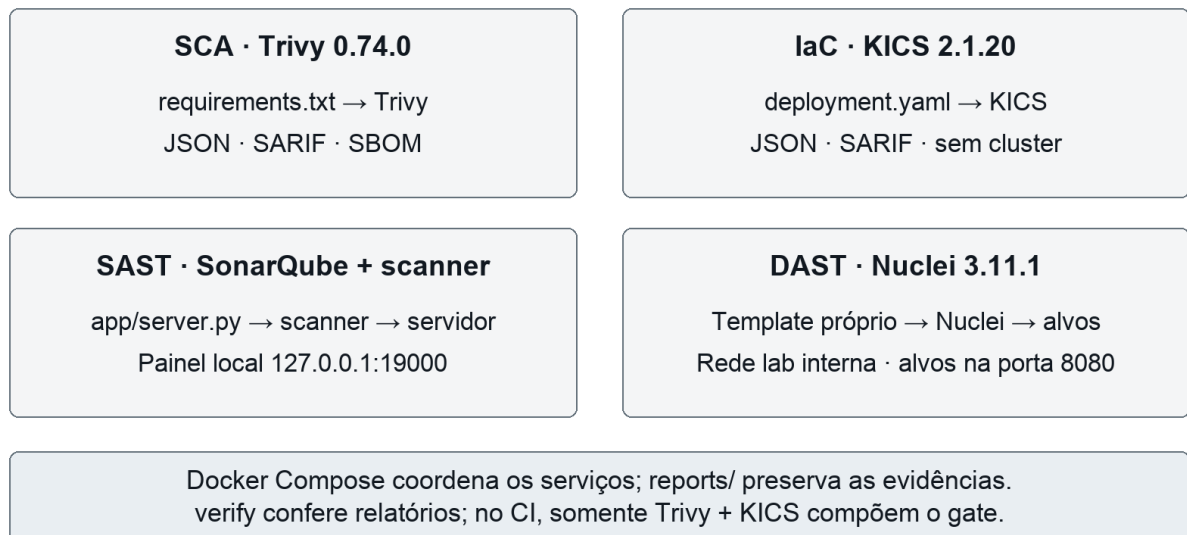
Para que resultados de ferramentas diferentes possam ser consolidados, adota-se o formato SARIF, padrão OASIS para intercâmbio de resultados de análise

estática (OASIS, 2020). Trivy e KICS exportam SARIF neste projeto, o que viabiliza integração futura com painéis como o GitHub Code Scanning.

3.2 VISÃO GERAL DA ARQUITETURA

A solução é inteiramente containerizada e orquestrada por um único arquivo `docker-compose.yml`, com o projeto nomeado `cp01-devsecops`. Os serviços são organizados em perfis (*profiles*) para que cada grupo de contêineres seja iniciado somente quando necessário: `tools` para os scanners e o verificador, `demo` para as duas instâncias da aplicação e `sonar` para o servidor e o scanner SonarQube. A Figura 2 apresenta a arquitetura.

Figura 2 – Arquitetura da solução CP01-DevSecOps



Fonte: Elaborado pelos autores (2026).

Duas redes Docker separam as responsabilidades. A rede `lab` é declarada com `internal: true`: as aplicações-alvo e o Nuclei se comunicam entre si, mas não há portas publicadas no hospedeiro nem rota para a internet. A rede `sonar` é do tipo *bridge* e publica apenas o painel do SonarQube no endereço de loopback `127.0.0.1:19000`. Os scanners Trivy e KICS rodam como contêineres efêmeros (`docker compose run --rm`), montando as *fixtures* em modo somente leitura (`/workspace:ro`) e gravando os resultados em `/reports`, mapeado para a pasta `reports/` do repositório.

3.3 STACK TECNOLÓGICA

Quadro 3 – Stack tecnológica e versões fixadas

Camada	Tecnologia	Versão no laboratório	Função
Linguagem da aplicação	Python (imagem python:3.12-slim)	3.12	Servidor HTTP didático e scripts de automação
Motor de templates	Jinja2 / MarkupSafe	3.1.4 → 3.1.6 / 3.0.3	Renderização em SandboxedEnvironment
Containerização	Docker Engine e Docker Compose	Engine 29.x; Compose 5.x	Orquestração local e isolamento
SAST	SonarQube Community Build + SonarScanner CLI	26.9.0.129388 / 12.2.0.4256_8.1.0	Análise do código próprio
SCA	Trivy (Aqua Security)	0.74.0	Vulnerabilidades em dependências; SBOM
IaC	KICS (Checkmarx)	2.1.20	Regras para manifestos Kubernetes
DAST	Nuclei (ProjectDiscovery)	3.11.1	Template HTTP próprio contra alvos locais
CI	GitHub Actions (runner ubuntu-24.04)	–	Gate automatizado Trivy + KICS
Formatos	JSON, JSONL, SARIF 2.1.0, CycloneDX	–	Evidências e intercâmbio

Fonte: Elaborado pelos autores (2026).

3.4 FERRAMENTAS DE SEGURANÇA

Para cada ferramenta registram-se identificação, fundamento técnico, uso no projeto, integrações e avaliação crítica. Distingue-se sempre o **suporte documentado** pela ferramenta daquilo que foi **efetivamente implementado** neste laboratório.

3.4.1 SonarQube Community Build (SAST)

O SonarQube é mantido pela SonarSource, com plataforma predominantemente Java e histórico de versões desde 2007. A plataforma permanece LGPLv3; desde 29/11/2024 os analisadores incluídos estão sujeitos à SSALv1, licença de código disponível com condições próprias. Portanto, não se caracteriza toda a distribuição apenas como LGPL ou inteiramente open source (SonarSource, [s.d.]d; [s.d.]b). O SonarScanner aplica os analisadores de linguagem localmente e envia o relatório ao servidor, que processa issues, métricas, quality gate e histórico (SonarSource, [s.d.]a).

No laboratório, o servidor 26.9.0.129388 utiliza H2 para avaliação; o scanner 12.2.0.4256_8.1.0 usa AMD64 com emulação no Mac ARM64. setup_sonar.py prepara credenciais locais, e run_sonar.py exporta JSON para reports/sonarqube/. A

regra S4790 detectou MD5 e foi encerrada após SHA-256 no commit c736a60; S5332, referente a HTTP, permaneceu aberta. O banco recém-criado não contém essa sequência histórica.

Avaliação e escolha: adequado para acompanhar qualidade do código próprio e evolução das issues; exige servidor e inicialização, por isso ficou fora do LAB de 12 minutos e do CI implementado. O Compose limita a memória do servidor a 3 GB: isso não é reserva nem medição de consumo. A análise de Ronaldo levou 15,907 s, com servidor preparado. MD5 foi detectado corretamente, mas seu impacto como checksum não foi demonstrado; HTTP é uma condição real aceita apenas no isolamento didático. Não se estimou taxa geral de falsos positivos.

Atividade consultada em 27/09/2026: release 26.9.0.129388 de 02/09, igual à usada; commit recente c121c6a7 de 25/09; 11.031 estrelas e 2.234 forks no GitHub. Esses indicadores mostram participação e atualização, sem medir eficácia de detecção (SonarSource, 2026).

Técnica e ajuste: os analisadores percorrem a representação sintática/semântica do código e aplicam regras específicas da linguagem. Quality profiles permitem ativar e parametrizar regras; criar um analisador/regra própria exige suporte específico da linguagem, não apenas editar um YAML. Issues podem receber classificação de falso positivo após triagem. Não foram criadas regras nem suprimidos resultados aqui. Não se presume que recursos avançados de outras edições, como análises especializadas de fluxo, estejam cobertos por este ensaio.

Integrações: o scanner pode ser chamado no CI e o SonarQube for IDE fornece feedback no editor. Um hook local de pre-commit seria uma adaptação do grupo, não implementada. A Community Build admite importar issues externas, inclusive SARIF; isso não equivale a exportar nativamente os achados desta análise para Code Scanning. Aqui foi usada a API JSON. O DefectDojo lista parser SonarQube, mas importação e compatibilidade do formato não foram testadas (SonarSource, [s.d.]f; [s.d.]c; DefectDojo, [s.d.]).

3.4.2 Trivy (SCA)

O Trivy, da Aqua Security, é escrito em Go e usa Apache 2.0. Seu repositório foi criado em 2019 (Aqua Security, 2026). No SCA, identifica pacotes e versões em manifestos e consulta a base local de advisories: essa associação detecta dependências conhecidamente afetadas, sem provar que uma falha seja explorável.

A ferramenta também oferece análise de imagens, configurações, segredos e licenças; tais capacidades não foram todas acionadas neste projeto (Aqua Security, [s.d.]c).

No laboratório foi usado o subcomando `trivy fs` com `--scanners vuln`. A base foi preparada previamente com `--download-db-only` no volume `trivy-cache` e reutilizada com `--skip-db-update --offline-scan`. O subcomando `convert` gera, a partir do JSON, a visualização em tabela, o SARIF e o SBOM CycloneDX, além de aplicar o gate com `--severity HIGH,CRITICAL --exit-code 1`.

Avaliação e escolha: selecionado pela análise local de dependências e exportação de SBOM, complementando o exame de código próprio feito pelo SAST. Nas medições iniciais de 25/09, com cache aquecido, cada scan levou menos de um segundo. A base muda independentemente da imagem fixada; o `requirements.txt` não cobre os pacotes do sistema operacional nem detecta falhas ainda desconhecidas. A CVE principal revisada foi verdadeiro positivo quanto à versão e ao contexto de uso; as outras duas não receberam triagem individual aprofundada.

Atividade consultada em 27/09/2026: release v0.74.0 de 14/08, igual à usada; commit ae561f8c de 25/09; 38.091 estrelas e 715 forks (Aqua Security, 2026). Não se confunde atualização do programa com atualização da base de vulnerabilidades.

Regras e integrações: a correspondência SCA usa advisories, enquanto verificações personalizadas de configuração podem usar Rego. `.trivyignore` e políticas de ignore permitem exceções justificadas; nenhuma foi aplicada no LAB. A CLI integra-se ao CI, a tarefas de IDE e a hooks locais/comunitários de pre-commit. Apenas o CI foi implementado. O SARIF gerado pode ser enviado ao Code Scanning e há parser Trivy no DefectDojo; esses destinos não foram configurados (Aqua Security, [s.d.]b; [s.d.]a; DefectDojo, [s.d.]).

3.4.3 KICS (IaC)

O KICS, da Checkmarx, foi lançado publicamente em 25/02/2021; o repositório remonta a 2020. O motor é escrito em Go e as queries em Rego, linguagem do Open Policy Agent, sob Apache 2.0 (Checkmarx, 2021; 2026). Os arquivos IaC são interpretados e normalizados para consulta por políticas; não há implantação nem execução do manifesto. Queries próprias reúnem `query.rego`, metadados e casos positivos/negativos de teste (Checkmarx, [s.d.]c).

No laboratório, kics scan usa `--type Kubernetes, --report-formats json,sarif` e `--fail-on high,critical`. O código 50 indica achados no limiar configurado; neste caso foram dois HIGH. Código 0 não significa ausência de todos os avisos. O executor cruza o código retornado com as contagens e considera divergências erro operacional (Checkmarx, [s.d.]a).

Avaliação e escolha: o KICS permitiu validar infraestrutura antes de criar um cluster. Os dois HIGH revisados foram verdadeiros positivos na configuração; o scanner não verifica políticas externas de admissão nem o estado real de um cluster. Avisos LOW relativos a outros objetos exigem contexto. Os scans iniciais de 25/09 duraram 1,770 s e 1,291 s. Foi mantida a versão 2.1.20 validada, embora a pesquisa tenha encontrado release posterior.

Atividade consultada em 27/09/2026: release mais recente v2.2.0 de 17/09 e commit 453066aa dessa data; 2.709 estrelas e 384 forks. O laboratório permanece em 2.1.20, versão efetivamente executada (Checkmarx, 2026).

Regras e integrações: queries próprias podem ser carregadas pelo scanner; `--exclude-queries` e filtros permitem excluir verificações após triagem documentada. Nenhuma exclusão foi usada. Há integrações documentadas com CI, extensão VS Code e hooks de pre-commit. Aqui somente o CI Trivy/KICS e a exportação SARIF foram implementados; Code Scanning e o parser KICS do DefectDojo são possibilidades ainda não integradas (Checkmarx, [s.d.]a; [s.d.]d; DefectDojo, [s.d.]j).

3.4.4 Nuclei (DAST)

O Nuclei é mantido pela ProjectDiscovery, escrito em Go e licenciado sob MIT; seu repositório foi criado em 2020 (ProjectDiscovery, 2026). Templates YAML definem metadados, requisições e matchers sobre respostas. Isso permite verificar condições específicas de uma aplicação em execução; não constitui cobertura automática de toda a lógica de negócio. O grupo criou exclusivamente o template `debug-exposure.yaml` (ProjectDiscovery, [s.d.]a; [s.d.]b).

O template próprio envia GET /debug e exige status 200 e dois marcadores simultâneos. A configuração atual usa `-duc`, `-ni` e `DISABLE_NUCLEI_TEMPLATES_PUBLIC_DOWNLOAD=true`, além da seleção explícita do template e da rede interna. A variável foi acrescentada após diagnosticar a espera por templates públicos; os relatórios pessoais anteriores de Ronaldo foram preservados.

Avaliação e escolha: adequado para confirmar dinamicamente a exposição de /debug, complementar à inspeção de arquivos. A severidade MEDIUM foi atribuída pelo grupo; os dados são fictícios. O resultado zero no corrigido comprova apenas esse template. Não houve campanha abrangente de falsos positivos. Os tempos pessoais de 180,892 s e 180,781 s incluem a espera inicial; testes diagnósticos assistidos posteriores terminaram abaixo de um segundo por cenário, sem substituir as evidências pessoais.

Atividade consultada em 27/09/2026: release v3.11.1 de 08/08, igual à usada; commit c5611b9a de 26/09; 31.563 estrelas e 3.899 forks (ProjectDiscovery, 2026). O catálogo público é independente do template próprio empregado.

Regras e integrações: criar um teste exige definir id, info, author, protocolo e matchers; exclusões por ID/tag são possíveis, mas não foram usadas. A CLI pode executar em CI após disponibilizar um alvo autorizado. Como proposta local, uma tarefa de IDE ou hook poderia validar templates com -validate; DAST completo não foi colocado no pre-commit. A ferramenta oferece JSONL e SARIF, mas o projeto preserva JSONL. Code Scanning exigiria exportação/upload; DefectDojo dispõe de parser Nuclei. Nenhuma dessas integrações foi executada aqui (ProjectDiscovery, [s.d.]a; [s.d.]b; DefectDojo, [s.d.]).

3.4.5 Quadro comparativo

Quadro 4 – Comparativo das ferramentas utilizadas

Critério	SonarQube CE	Trivy	KICS	Nuclei
Categoria	SAST	SCA (e outros)	IaC	DAST
Mantenedor / licença	SonarSource / LGPLv3; analisadores SSALv1	Aqua Security / Apache-2.0	Checkmarx / Apache-2.0	ProjectDiscovery / MIT
Entrada no projeto	app/server.py	requirements.txt	deployment.yaml	Endpoint /debug
Precisa da aplicação rodando?	Não (precisa de servidor)	Não	Não	Sim
Formatos usados	API JSON	JSON, SARIF, CycloneDX	JSON, SARIF	JSONL
No gate do CI?	Não	Sim	Sim	Não
Custo operacional	Maior: servidor limitado a 3 GB	Baixo	Baixo	Baixo
Principal limite	Triagem contextual	Base muda com o tempo	Não vê o cluster	Cobre só o template

Fonte: Elaborado pelos autores (2026).

3.5 APLICAÇÃO-ALVO E API

A aplicação-alvo (`app/server.py`) é um servidor HTTP em Python, baseado em `ThreadingHTTPServer` da biblioteca padrão, que escuta na porta 8080 dentro do contêiner. O modo de operação é definido pela variável `LAB_MODE`, que diferencia os estados vulnerável e corrigido. O Quadro 5 descreve a API.

Quadro 5 – Endpoints da aplicação-alvo

Método e rota	Comportamento	Observação de segurança
GET / e GET /health	Retorna JSON com nome do projeto, status ok e modo.	Usado pelo <i>healthcheck</i> do contêiner.
GET /debug	No modo vulnerável, retorna configuração interna fictícia com o marcador <code>cp01-debug-exposure</code> ; no corrigido, 404.	Simula CWE-497 (exposição de informação do sistema).
POST /preview	Corpo JSON de 1 a 4.096 bytes; template opcional com padrão. Renderiza em sandbox com autoescape; retorna resultado e SHA-256.	Contexto que torna o Jinja2 afetado relevante (CWE-1336).
Outros caminhos GET/POST	Retornam 404 em JSON. Métodos não implementados podem retornar 501 pela biblioteca padrão.	nosniff nas respostas geradas pelo handler; não generalizar às respostas automáticas da biblioteca.

Fonte: Elaborado pelos autores (2026).

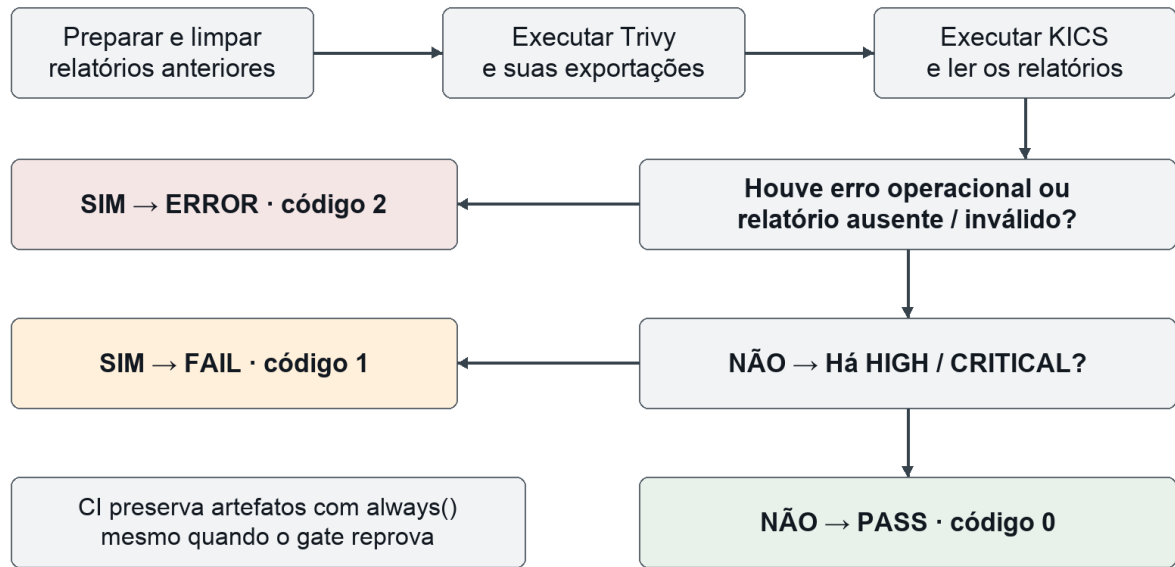
3.6 PERSISTÊNCIA E ARTEFATOS DE DADOS

A persistência se limita a volumes Docker e relatórios. `trivy-cache` guarda a base; os volumes `sonar-data`, `sonar-extensions` e `sonar-logs` preservam dados do servidor, e `sonar-scanner-cache` guarda o cache do scanner. Trivy, KICS e Nuclei possuem artefatos por cenário; SonarQube grava diretamente em `reports/sonarqube/` e mantém evidência histórica em `reports/sonarqube-antes/`. O resumo `reports/gate/<cenário>/summary.json` registra tempos, contagens, política, erros e hashes dos arquivos.

3.7 PIPELINE DE INTEGRAÇÃO CONTÍNUA

O workflow `.github/workflows/security.yml`, denominado *Security gate*, é disparado em *push* para `main`, em *pull requests* e manualmente (`workflow_dispatch`), quando é possível escolher o cenário. Ele concede ao token apenas `contents: read`, fixa as *actions* por hash de commit e limita o job a 15 minutos. A Figura 3 detalha o fluxo.

Figura 3 – Fluxo do gate de segurança no GitHub Actions



Fonte: Elaborado pelos autores (2026).

O executor chama Trivy e depois KICS, validando relatórios e priorizando erros operacionais sobre achados. O upload com `if: always()` preserva evidências mesmo diante de reprovação. Não há deploy nem upload para Code Scanning. A falha do job somente impede merge se houver proteção/check obrigatório correspondente; essa configuração do GitHub não foi auditada.

4 IMPLEMENTAÇÃO E MANIFESTOS

Os caminhos desta seção são relativos à raiz do repositório. A base analisada é o commit `af4da9ba33d238d41ca90625498a0f42b7eeaca6`, na branch `feat/validacao-ronaldo` (Grupo 2, 2026). Trechos indicados como ilustrativos não substituem os arquivos completos versionados. Referência imutável: <https://github.com/MaduKa01/CP01-DevSecOps/tree/af4da9ba33d238d41ca90625498a0f42b7eeaca6>.

4.1 PRÉ-REQUISITOS DO AMBIENTE

- a) Git instalado e disponível no terminal (`git --version`);
- b) Docker Desktop (Windows ou macOS) ou Docker Engine (Linux) com o plugin Compose; o comando `docker version` deve exibir as seções *Client* e *Server*;
- c) no Windows, WSL 2 habilitado e contêineres Linux selecionados (`wsl --status`);
- d) Python 3.10 ou superior apenas para os scripts complementares do grupo; o roteiro da turma usa Python dentro do contêiner;
- e) internet e espaço para baixar imagens e base; memória disponível para Docker, alvos, servidor SonarQube e scanner. O limite de 3 GB do servidor não representa o consumo total do ambiente;

Não é necessário instalar Kubernetes, Java nem os scanners no sistema hospedeiro, e não se deve instalar a versão vulnerável do Jinja2 fora dos contêineres.

4.2 OBTENÇÃO DO PROJETO E PREPARAÇÃO

Código-fonte 1 – Clonagem, conferência do ambiente e preparo dos downloads

```
git clone https://github.com/MaduKa01/CP01-DevSecOps.git
cd CP01-DevSecOps
git switch feat/validacao-ronaldo
git rev-parse HEAD
docker version
docker compose config --quiet

# Downloads antecipados (fora do cronômetro do laboratório)
docker compose pull trivy kics verify
docker compose run --rm trivy image --download-db-only
```

Fonte: Repositório CP01-DevSecOps (Grupo 2, 2026).

4.3 ORQUESTRAÇÃO COM DOCKER COMPOSE

O Código-fonte 5 ilustra o endurecimento compartilhado pelos alvos e o serviço Trivy. As imagens externas de ferramentas e a base Python são fixadas por digest; as duas imagens da aplicação são construídas localmente e identificadas por tags. Executar o Compose completo versionado, não o trecho abreviado.

O trecho ilustrativo correspondente encontra-se no Apêndice C; o arquivo completo está no repositório.

4.4 DOCKERFILE DA APLICAÇÃO

Um único Dockerfile gera as duas imagens da aplicação por meio do argumento de build `VARIANT`, que seleciona qual `requirements.txt` será instalado. A imagem cria um usuário sem privilégios (UID 10001) e define um `healthcheck` usado pelo Compose com a opção `--wait`.

O trecho ilustrativo correspondente encontra-se no Apêndice C; o arquivo completo está no repositório.

4.5 MANIFESTOS KUBERNETES VULNERÁVEL E CORRIGIDO

O arquivo `deployment.yaml` é um artefato de estudo analisado estaticamente e **não deve ser aplicado em cluster**. Ambos os estados já adotam boas práticas no nível do pod (usuário não root, perfil `seccomp` `RuntimeDefault`, remoção de todas as capabilities e limites de recursos). A diferença está concentrada em três propriedades do `securityContext` do contêiner, apresentadas no Quadro 6 e no Código-fonte 7.

Quadro 6 – Diferenças entre os estados vulnerável e corrigido

Arquivo / propriedade	Vulnerável	Corrigido	Efeito no scanner
requirements.txt – Jinja2	3.1.4	3.1.6	Remove as três CVEs no Trivy
privileged	true	false	Remove um HIGH no KICS
allowPrivilegeEscalation	true	false	Remove um HIGH no KICS
readOnlyRootFilesystem	false	true	Remove um LOW no KICS

Fonte: Elaborado pelos autores (2026).

O trecho ilustrativo correspondente encontra-se no Apêndice C; o arquivo completo está no repositório.

4.6 TEMPLATE NUCLEI

O trecho ilustrativo correspondente encontra-se no Apêndice C; o arquivo completo está no repositório.

4.7 VARIÁVEIS DE AMBIENTE E CONFIGURAÇÃO

Quadro 7 – Variáveis de ambiente e argumentos de build

Variável	Onde	Valor / finalidade
LAB_MODE	app-vulneravel, app-corrigido	vulneravel ou corrigido; habilita ou não o endpoint /debug
VARIANT (build arg)	app/Dockerfile	Seleciona a <i>fixture</i> de dependências instalada na imagem
PYTHON_IMAGE (build arg)	app/Dockerfile	Imagem base fixada por digest
PYTHONDONTWRITEBYTECODE, PYTHONUNBUFFERED	app/Dockerfile	1; evita arquivos .pyc e <i>buffer</i> de logs
DISABLE_NUCLEI_TEMPLATES_PUBLIC_DOWNLOAD	nuclei	true; impede download de templates públicos
SONAR_TELEMETRY_ENABLE	sonarqube	false; desativa telemetria
SONAR_HOST_URL	sonar-scanner	http://sonarqube:9000 (rede interna sonar)
SONAR_TOKEN, SONAR_ADMIN_PASSWORD	.env local	Gerados por setup_sonar.py; nunca versionados
DISABLE_CRASH_REPORT	kics	Configuração de relatório de falhas do KICS
LAB_VARIANT	GitHub Actions	Cenário analisado; padrão corrigido

Fonte: Elaborado pelos autores (2026).

A configuração do SonarQube é complementada pelo arquivo `sonar-project.properties`, que define a chave do projeto (`cp01-devsecops-grupo2`), a pasta de fontes (`app`), a versão do Python (3.12) e a codificação UTF-8.

4.8 GUIA DE EXECUÇÃO

O roteiro Trivy/KICS usa os comandos do Apêndice B, cada qual em uma única linha lógica, compatível com Zsh/Bash e PowerShell. Quebras visuais na página não exigem inserir barra invertida. Conferir imediatamente o código com `echo $?` no macOS/Linux ou `$LASTEXITCODE` no PowerShell. O KICS vulnerável deve retornar 50; os gates corrigidos, 0. Gerar o comprovante após concluir ambos os cenários.

Na automação, `run_scan.py` vulneravel retorna 1, corrigido retorna 0, e erro operacional retorna 2. Para os scripts locais, usar `python3` no macOS/Linux e `python` no Windows, conforme a instalação. Antes do SAST, aguardar o servidor e repetir `setup_sonar.py` se ele ainda estiver iniciando. Uma instalação nova analisa o código já corrigido e não reproduz a issue MD5 encerrada; consultar os relatórios históricos para esse antes/depois.

4.9 WORKFLOW DO GITHUB ACTIONS

O workflow `security.yml` (Código-fonte 4, Apêndice B) implementa a Figura 3. Suas decisões de configuração mais relevantes são: permissão mínima `contents: read` para o token; *actions* fixadas por hash de commit, e não por tag mutável; variável `LAB_VARIANT` com padrão `corrigido`, que só muda por escolha explícita no disparo manual; execução prévia de `test_gate.py`, que impede falso verde; e upload condicional `if: always()` dos relatórios com retenção de 30 dias, identificados pelo cenário e pelo número do run.

5 TESTES E VALIDAÇÃO

5.1 PLANO DE TESTES

O plano verifica detecção, correção sem supressões, estados do gate e reprodução do LAB em outra máquina. O Quadro 8 descreve seus níveis; a seção 5.3 identifica a evidência de cada caso.

Quadro 8 – Níveis do plano de testes

Nível	Objetivo	Instrumento	Critério de aceite
Unitário do gate	Garantir que erro não gere falso verde	scripts/test_gate.py com unittest.mock	2 testes aprovados
Funcional das ferramentas	Detecção e correção nos dois estados	Trivy, KICS, Nuclei, SonarQube	Contagens esperadas do Quadro 10
Integração contínua	Vermelho e verde reais no runner Linux	GitHub Actions	FAIL no vulnerável; PASS no corrigido
Reprodução independente	Execução por outro integrante em outro sistema	Roteiro LAB.md e comprovante	Mesmas contagens; comprovante válido

Fonte: Elaborado pelos autores (2026).

5.2 CENÁRIOS E CASOS DE TESTE

Quadro 9 – Casos de teste e resultados esperados

ID	Cenário	Resultado esperado	Requisito
CT01	Trivy no estado vulnerável	3 MEDIUM no Jinja2 3.1.4; código 0 no gate (sem HIGH)	RF01
CT02	Trivy no estado corrigido	0 vulnerabilidades; código 0	RF01, RF07
CT03	KICS no estado vulnerável	2 HIGH, 1 MEDIUM, 6 LOW; código 50	RF02, RF06
CT04	KICS no estado corrigido	0 HIGH/CRITICAL, 1 MEDIUM, 5 LOW; código 0	RF02, RF07
CT05	run_scan.py vulneravel	Status FAIL, 2 bloqueantes, código 1	RF06, RN04
CT06	run_scan.py corrigido	Status PASS, 0 bloqueantes, código 0	RF07, RN04
CT07	Relatório ausente após saída 0 do scanner	Status ERROR, código 2	RF08, RN05
CT08	Variante inválida	Rejeição antes da execução	RF08
CT09	Nuclei nos dois alvos	1 correspondência no vulnerável; 0 no corrigido	RF04
CT10	SonarQube antes/depois no histórico preservado	S4790 encerrada; S5332 aberta; 0 hotspots	RF03
CT11	Comprovante antigo: teste negativo sem registro localizado	Rejeição de relatórios com mais de 2 h	RF10
CT12	Reprodução Trivy/KICS em Windows e macOS	Contagens idênticas às de referência	RNF02

Fonte: Elaborado pelos autores (2026).

5.3 RESULTADOS OBTIDOS

As execuções de scanners, gates e testes unitários registradas produziram os resultados do Quadro 10. CT01–CT06 são sustentados pelos relatórios Trivy/KICS/gate; CT07–CT08 pelos dois testes unitários; CT09 pelos relatórios Nuclei; CT10 pelo histórico SonarQube; CT12 pelos relatos pessoais e comprovantes. Para CT11, o controle existe no código, mas não foi localizada evidência da execução negativa com relatório de mais de duas horas.

Quadro 10 – Resultados consolidados por ferramenta

Ferramenta	Estado vulnerável	Estado corrigido
Trivy 0.74.0	3 MEDIUM: CVE-2024-56201, CVE-2024-56326, CVE-2025-27516 (Jinja2 3.1.4)	0 vulnerabilidades nas dependências declaradas (Jinja2 3.1.6)
KICS 2.1.20	2 HIGH, 1 MEDIUM, 6 LOW; código 50	0 HIGH/CRITICAL, 1 MEDIUM, 5 LOW; código 0
Nuclei 3.11.1	1 correspondência MEDIUM em /debug	0 correspondências (/debug retorna 404)
SonarQube CE	S4790 (MD5) e S5332 (HTTP) abertas	S4790 encerrada; S5332 aberta; 0 hotspots
Gate (Trivy + KICS)	FAIL, 2 bloqueantes, código 1	PASS, 0 bloqueantes, código 0

Fonte: Relatórios em reports/ do repositório CP01-DevSecOps (Grupo 2, 2026).

No GitHub Actions, as duas execuções de referência usaram o mesmo commit (`c736a60`), as mesmas imagens e a mesma política, variando apenas a *fixture*: o run 36167707788 (cenário corrigido) terminou em SUCCESS e o run 36167909434 (cenário vulnerável) terminou em FAILURE na etapa do gate, com `blocking_findings: 2` e lista de erros vazia – o que confirma que a reprovação decorreu da política de segurança, e não de indisponibilidade de scanner. Em ambos os casos a etapa de artifacts foi concluída.

As medições iniciais de 25/09/2026, preservadas em evidencias/metricas.md, registraram Trivy em 0,587/0,452 s, KICS em 1,770/1,291 s e executor completo em 3,244/2,723 s (vulnerável/corrigido), com cache aquecido. A base foi atualizada às 13:09:04 UTC daquele dia. Os relatórios locais foram renovados em 26/09. São medições individuais de fixtures pequenas, não um benchmark.

5.4 ANÁLISE DOS ACHADOS PRINCIPAIS

Foram selecionados três achados para análise aprofundada, conforme exigido pela avaliação. O Quadro 11 resume evidência, CWE, triagem e correção de cada um.

Quadro 11 – Triagem dos três achados principais

Achado	Evidência e CWE	Triagem	Correção e comprovação
CVE-2025-27516 no Jinja2	requirements.txt linha 1; Trivy; CWE-1336	Verdadeiro positivo: versão afetada e uso de templates do usuário em sandbox; exploração não realizada	Atualizar para 3.1.6; achado ausente no relatório corrigido
Container Is Privileged	privileged: true (linha 31); KICS; CWE-269	Verdadeiro positivo na configuração declarada	privileged: false; regra ausente no corrigido
Privilege Escalation Allowed	allowPrivilegeEscalation: true (linha 32); KICS; CWE-269	Verdadeiro positivo na configuração declarada	false; segundo HIGH removido

Fonte: Adaptado de evidencias/achados.md (Grupo 2, 2026).

CVE-2025-27516. Segundo o *advisory* publicado pelos mantenedores em 5 de março de 2025, um invasor que controle o conteúdo de um template pode contornar o *sandbox* do Jinja2 por meio do filtro `|attr`, que permitia acessar o método `format` de strings e, assim, executar código Python arbitrário (Pallets, 2025b). O endpoint `/preview` do projeto aceita templates enviados pelo usuário e usa `SandboxedEnvironment`, que é exatamente o cenário de uso relevante. O achado foi classificado como verdadeiro positivo quanto à exposição, sem execução de prova de conceito. A correção consistiu em atualizar para a versão 3.1.6, que corrige o problema (Pallets, 2025a). A CWE-1336 descreve a neutralização imprópria de elementos especiais em motores de template (MITRE, [s.d.]a).

Container privilegiado. A regra `dd29336b-fe57-445b-a26e-e6aa867ae609` do KICS sinaliza contêineres executados com `privileged: true` (Checkmarx, [s.d.]b). A documentação do Kubernetes esclarece que um contêiner privilegiado recebe praticamente as mesmas capacidades de processos no hospedeiro, anulando boa parte do isolamento (Kubernetes, [s.d.]). Trata-se de gestão imprópria de privilégios, CWE-269 (MITRE, [s.d.]b). O achado é verdadeiro positivo na configuração declarada; não foi criado pod para demonstrar impacto.

Escalada de privilégios permitida. A regra `5572cc5e-1e4c-4113-92a6-7a8a3bd25e6d` sinaliza `allowPrivilegeEscalation: true` (Checkmarx, [s.d.]e). Essa propriedade controla se um processo pode obter mais privilégios que seu processo pai, por exemplo por binários `setuid` (Kubernetes, [s.d.]). É um controle distinto de `privileged`, embora relacionado; por isso ambos foram corrigidos em conjunto.

Nos três achados principais foram classificados zero falsos positivos (0/3). Esse número refere-se apenas à amostra revisada e não estima a taxa global de falsos positivos das ferramentas. No SonarQube, o alerta S4790 recebeu triagem contextual: o MD5 era usado apenas como *checksum* sem função de autenticação, de modo que o algoritmo foi detectado corretamente, mas o impacto de segurança não foi demonstrado; ainda assim, a troca por SHA-256 foi adotada por ser simples. O alerta S5332 (HTTP) permanece aberto como condição real, aceita apenas no isolamento didático da rede interna.

5.5 VALIDAÇÃO EM AMBIENTES INDEPENDENTES

Cauê reproduziu Trivy/KICS no Windows e Ronaldo no macOS, com assistência do Codex e supervisão/autorização dos integrantes. A independência refere-se à pessoa e à máquina, não à ausência de assistência. Nuclei e SonarQube têm evidências complementares no Mac; não foi comprovada reprodução dessas duas ferramentas no Windows.

Quadro 12 – Comparação das validações independentes

Item	Cauê – Windows	Ronaldo – macOS
Data	25/09/2026	26/09/2026
Sistema	Windows 64-bit, WSL 2.7.14	macOS 26.5.2, ARM64
Docker Desktop / Engine	4.92.0 / 29.8.0	4.63.0 / 29.2.1
Compose / Git / Python	5.5.1 / 2.55.0 / 3.14.7	5.0.2 / 2.50.1 / 3.14.7
Trivy vulnerável → corrigido	3 MEDIUM → 0	3 MEDIUM → 0
KICS vulnerável	2 HIGH, 1 MEDIUM, 6 LOW	2 HIGH, 1 MEDIUM, 6 LOW
KICS corrigido	0 HIGH, 1 MEDIUM, 5 LOW	0 HIGH, 1 MEDIUM, 5 LOW
Testes do gate	2/2 aprovados	2/2 aprovados
Tempo registrado	118,404 s de comandos automatizados	1.136 s (18 min 56 s), com intervalos

Fonte: Adaptado de evidencias/validacao-kaue.md e evidencias/validacao-ronaldo.md (Grupo 2, 2026).

As contagens Trivy/KICS coincidiram. Os 118,404 s de Cauê correspondem ao fluxo automatizado; os 1.136 s de Ronaldo incluem pausas. Seus executores completos registraram 25,535 s no Windows e soma das etapas de 3,144/2,539 s no Mac. Como caches e métodos diferem, não se compara desempenho entre sistemas. Esses registros não comprovam ensaio explicado de 12 minutos.

5.6 DIFICULDADES ENCONTRADAS E TRATAMENTO

Quadro 13 – Ocorrências registradas e orientações preventivas

Ocorrência / orientação	Causa	Tratamento
Registrada: teste falhava no Windows	Dependência de executável Unix no teste	Simulação com unittest.mock (PR #1)
Registrada: espera Nuclei de cerca de 180 s	Busca automática de templates públicos	Desativar download público; diagnóstico assistido < 1 s por cenário. Relatórios pessoais anteriores preservados.
Orientação preventiva: Docker exibe Client, mas não Server	Motor não iniciado ou WSL pendente de reinicialização	Abrir o Docker Desktop; concluir reinicialização
Orientação preventiva: Trivy sem banco em modo offline	Base não preparada	Executar --download-db-only com internet
Aviso observado no convert	Aviso de apresentação	Conferir o JSON; o scan não está ausente
Orientação preventiva: SonarQube não responde	Inicialização lenta do servidor	Aguardar <i>SonarQube is operational</i> nos logs e repetir o setup
Orientação preventiva: Comprovante rejeita relatório	Relatório com mais de 2 h	Refazer os scans; não alterar horários

Fonte: Adaptado de docs/GUIA-PARA-DOCUMENTACAO.md (Grupo 2, 2026).

6 CONSIDERAÇÕES FINAIS

O gate Trivy/KICS reprovou o estado vulnerável e aprovou a correção sem supressões; a reprodução Windows/macOS produziu contagens coincidentes. Nuclei e SonarQube complementaram a análise no Mac. Os resultados sustentam a política aplicada e não demonstram ausência de risco.

6.1 GANHOS ALCANÇADOS

- a) as configurações examinaram código, dependências, manifesto e endpoint, com resultados complementares;
- b) o gate distinguiu achados bloqueantes de erros operacionais nos testes registrados;
- c) o LAB Trivy/KICS teve reprodução documentada em duas máquinas;
- d) os alvos da aplicação permaneceram isolados, com privilégios reduzidos;
- e) relatórios JSON, SARIF, SBOM e execuções de CI permitem auditar os resultados.

6.2 LIMITAÇÕES

As fixtures pequenas não representam desempenho em produção. O SCA não cobriu pacotes do sistema operacional e sua base evolui; KICS não avaliou um cluster, e Nuclei executou apenas um template. Não houve exploração. Somente três achados receberam triagem completa; os avisos menores remanescentes não foram analisados individualmente. SonarQube usou H2 e ficou fora do CI.

6.3 TRABALHOS FUTUROS

Como evolução, propõe-se integrar Code Scanning/DefectDojo, analisar a imagem final, incluir SonarQube no CI com banco apropriado, antecipar controles no editor/pre-commit e ampliar políticas e templates. Para a entrega acadêmica, ainda precisam de comprovação o CT11 negativo, o ensaio explicado de até 12 minutos e o plano B audiovisual. A revisão não certifica publicação, apresentação ou prazos.

REFERÊNCIAS

AQUA SECURITY. **Custom checks**. [S. l.], [s.d.]a. Disponível em: <https://trivy.dev/docs/v0.72/guide/scanner/misconfiguration/custom/>. Acesso em: 27 set. 2026.

AQUA SECURITY. **Filtering**. [S. l.], [s.d.]b. Disponível em: <https://trivy.dev/docs/v0.72/guide/configuration/filtering/>. Acesso em: 27 set. 2026.

AQUA SECURITY. **Trivy documentation**. [S. l.], [s.d.]c. Disponível em: <https://trivy.dev/docs/latest/>. Acesso em: 27 set. 2026.

AQUA SECURITY. **Trivy: repositório e atividade do projeto**. GitHub, 2026. Consulta de release, commits e comunidade em 27 set. 2026. Disponível em: <https://github.com/aquasecurity/trivy>.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação**. Rio de Janeiro: ABNT, 2024.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023: informação e documentação: referências: elaboração**. Rio de Janeiro: ABNT, 2025.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520: informação e documentação: citações em documentos: apresentação**. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6024: informação e documentação: numeração progressiva das seções de um documento: apresentação**. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6027: informação e documentação: sumário: apresentação**. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6028: informação e documentação: resumo, resenha e resensão: apresentação**. Rio de Janeiro: ABNT, 2021.

CHECKMARX. **Checkmarx launches Infrastructure as Code scanning solution**. [S. l.], 25 fev. 2021. Disponível em: <https://checkmarx.com/press-releases/checkmarx-launches-infrastructure-as-code-scanning-solution-to-secure-cloud-native-applications/>. Acesso em: 27 set. 2026.

CHECKMARX. **Commands**. [S. l.], [s.d.]a. Disponível em: <https://docs.kics.io/latest/commands/>. Acesso em: 27 set. 2026.

CHECKMARX. **Container is privileged**. [S. l.], [s.d.]b. Disponível em: <https://docs.kics.io/latest/queries/kubernetes-queries/dd29336b-fe57-445b-a26e-e6aa867ae609/>. Acesso em: 27 set. 2026.

CHECKMARX. **Creating queries.** [S. I.], [s.d.]c. Disponível em: <https://docs.kics.io/latest/creating-queries/>. Acesso em: 27 set. 2026.

CHECKMARX. **Integrations.** [S. I.], [s.d.]d. Disponível em: <https://docs.kics.io/latest/integrations/>. Acesso em: 27 set. 2026.

CHECKMARX. **KICS: repositório de código-fonte.** [S. I.]: GitHub, 2026. Disponível em: <https://github.com/Checkmarx/kics>. Acesso em: 27 set. 2026.

CHECKMARX. **Privilege escalation allowed.** [S. I.], [s.d.]e. Disponível em: <https://docs.kics.io/latest/queries/kubernetes-queries/5572cc5e-1e4c-4113-92a6-7a8a3bd25e6d/>. Acesso em: 27 set. 2026.

CYCLONEDX. **Specification overview.** [S. I.]: OWASP Foundation, [s.d.]. Disponível em: <https://cyclonedx.org/specification/overview/>. Acesso em: 27 set. 2026.

DEFECTDOJO. **Supported tools.** [S. I.], [s.d.]. Disponível em: https://docs.defectdojo.com/supported_tools/parsers/. Acesso em: 27 set. 2026.

GITHUB. **GitHub Actions documentation.** [S. I.], [s.d.]. Disponível em: <https://docs.github.com/en/actions>. Acesso em: 27 set. 2026.

GRUPO 2. **CP01-DevSecOps: laboratório DevSecOps com SonarQube, Trivy, KICS e Nuclei.** São Paulo, 2026. Commit af4da9ba33d238d41ca90625498a0f42b7eeaca6. Disponível em: <https://github.com/MaduKa01/CP01-DevSecOps/tree/af4da9ba33d238d41ca90625498a0f42b7eeaca6>. Acesso em: 27 set. 2026.

HUMBLE, Jez; FARLEY, David. **Continuous delivery: reliable software releases through build, test, and deployment automation.** Upper Saddle River: Addison-Wesley, 2010.

KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John. **The DevOps handbook: how to create world-class agility, reliability, and security in technology organizations.** Portland: IT Revolution Press, 2016.

KUBERNETES. **Configure a security context for a pod or container.** [S. I.]: The Linux Foundation, [s.d.]. Disponível em: <https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>. Acesso em: 27 set. 2026.

MICROSOFT. **About parsing.** [S. I.], [s.d.]. Disponível em: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_parsing. Acesso em: 27 set. 2026.

MITRE. **CWE-1336: improper neutralization of special elements used in a template engine.** Bedford, [s.d.]a. Disponível em: <https://cwe.mitre.org/data/definitions/1336.html>. Acesso em: 27 set. 2026.

MITRE. **CWE-269: improper privilege management**. Bedford, [s.d.]b. Disponível em: <https://cwe.mitre.org/data/definitions/269.html>. Acesso em: 27 set. 2026.

MITRE. **CWE-497: exposure of sensitive system information to an unauthorized control sphere**. Bedford, [s.d.]c. Disponível em: <https://cwe.mitre.org/data/definitions/497.html>. Acesso em: 27 set. 2026.

OASIS. **Static Analysis Results Interchange Format (SARIF) version 2.1.0**. [S. I.]: OASIS Open, 2020. OASIS Standard. Disponível em: <https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html>. Acesso em: 27 set. 2026.

OWASP FOUNDATION. **OWASP Top 10:2021**. [S. I.], 2021. Disponível em: <https://owasp.org/Top10/2021/>. Acesso em: 27 set. 2026.

PALLETS. **Jinja changes: version 3.1.6**. [S. I.], 5 mar. 2025a. Disponível em: <https://jinja.palletsprojects.com/en/stable/changes/#version-3-1-6>. Acesso em: 27 set. 2026.

PALLETS. **Jinja sandbox breakout through attr filter selecting format method: GHSA-cpwx-vrp4-4pq7**. GitHub, 5 mar. 2025b. Disponível em: <https://github.com/pallets/jinja/security/advisories/GHSA-cpwx-vrp4-4pq7>. Acesso em: 27 set. 2026.

PROJECTDISCOVERY. **Nuclei: repositório de código-fonte**. [S. I.]: GitHub, 2026. Disponível em: <https://github.com/projectdiscovery/nuclei>. Acesso em: 27 set. 2026.

PROJECTDISCOVERY. **Running Nuclei**. [S. I.], [s.d.]a. Disponível em: <https://docs.projectdiscovery.io/opensource/nuclei/running>. Acesso em: 27 set. 2026.

PROJECTDISCOVERY. **Template structure**. [S. I.], [s.d.]b. Disponível em: <https://docs.projectdiscovery.io/templates/structure>. Acesso em: 27 set. 2026.

SONARSOURCE. **Analysis overview**. [S. I.], [s.d.]a. Disponível em: <https://docs.sonarsource.com/sonarqube-community-build/analyzing-source-code/analysis-overview>. Acesso em: 27 set. 2026.

SONARSOURCE. **Historical downloads**. [S. I.], [s.d.]b. Disponível em: <https://www.sonarsource.com/products/sonarqube/downloads/historical-downloads/>. Acesso em: 27 set. 2026.

SONARSOURCE. **Importing external issues**. [S. I.], [s.d.]c. Disponível em: <https://docs.sonarsource.com/sonarqube-community-build/analyzing-source-code/importing-external-issues>. Acesso em: 27 set. 2026.

SONARSOURCE. **License information**. [S. I.], [s.d.]d. Disponível em: <https://www.sonarsource.com/license/>. Acesso em: 27 set. 2026.

SONARSOURCE. **SonarQube Community Build documentation**. [S. I.], [s.d.]e. Disponível em: <https://docs.sonarsource.com/sonarqube-community-build/>. Acesso em: 27 set. 2026.

SONARSOURCE. **SonarQube for IDE**. [S. l.], [s.d.].f. Disponível em: <https://www.sonarsource.com/products/sonarqube/ide/>. Acesso em: 27 set. 2026.

SONARSOURCE. **SonarQube: repositório de código-fonte**. [S. l.]: GitHub, 2026. Disponível em: <https://github.com/SonarSource/sonarqube>. Acesso em: 27 set. 2026.

SOUPPAYA, Murugiah; MORELLO, John; SCARFONE, Karen. **Application container security guide**. Gaithersburg: National Institute of Standards and Technology, 2017. (NIST Special Publication 800-190). DOI: <https://doi.org/10.6028/NIST.SP.800-190>.

SOUPPAYA, Murugiah; SCARFONE, Karen; DODSON, Donna. **Secure Software Development Framework (SSDF) version 1.1: recommendations for mitigating the risk of software vulnerabilities**. Gaithersburg: National Institute of Standards and Technology, 2022. (NIST Special Publication 800-218). DOI: <https://doi.org/10.6028/NIST.SP.800-218>.

UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO. **Normalização de trabalhos**. [S. l.], [s.d.]. Disponível em: <https://biblioteca.ufes.br/normalizacao>. Acesso em: 27 set. 2026.

APÊNDICE A – DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

O Codex foi utilizado como assistente na elaboração de código e scripts, preparação de comandos e guias, pesquisa, diagnóstico, revisão e correção desta documentação. Os relatos de Cauê e Ronaldo registram execuções supervisionadas/autorizadas por eles; o diagnóstico da espera do Nuclei também recebeu assistência. As contagens foram confrontadas com relatórios JSON, resumos dos executores, testes unitários e registros de CI versionados. Não se atribui à IA autoria de uma execução humana nem se apresenta geração de texto como prova de teste. O escopo detalhado consta em USO-DE-IA.md. Nesta revisão documental, resultados históricos foram preservados; os scanners não foram executados novamente.

APÊNDICE B – COMANDOS DE EXECUÇÃO E WORKFLOW

Os comandos abaixo usam uma única linha lógica por comando; a quebra automática da página não é continuação de shell. No Windows, substituir python3 por python quando esse for o comando da instalação. Os arquivos completos e os relatórios pertencem ao commit identificado na seção 4.

Código-fonte 2 – Roteiro principal do laboratório (Trivy + KICS)

```
# 1. Versões
docker compose run --rm trivy --version
docker compose run --rm kics version

# 2. SCA - estado vulnerável
docker compose run --rm trivy fs --skip-db-update --offline-scan --scanners vuln --
format json --output /reports/trivy/vulneravel/results.json
/workspace/vulneravel/requirements.txt
docker compose run --rm trivy convert /reports/trivy/vulneravel/results.json

# 3. IaC - estado vulnerável (esperado: código 50)
docker compose run --rm kics scan -p /workspace/vulneravel/deployment.yaml --type
Kubernetes --report-formats json,sarif --output-path /reports/kics/vulneravel --
fail-on high,critical --no-progress --no-color

# 4. Estado corrigido (esperado: código 0 em ambos os gates)
docker compose run --rm trivy fs --skip-db-update --offline-scan --scanners vuln --
format json --output /reports/trivy/corrigido/results.json
/workspace/corrigido/requirements.txt
docker compose run --rm trivy convert --severity HIGH,CRITICAL --exit-code 1
/reports/trivy/corrigido/results.json
docker compose run --rm kics scan -p /workspace/corrigido/deployment.yaml --type
Kubernetes --report-formats json,sarif --output-path /reports/kics/corrigido --
fail-on high,critical --no-progress --no-color

# 5. Comprovante pessoal (rejeita relatórios com mais de 2 h)
docker compose run --rm verify python scripts/comprovante.py
```

Fonte: Repositório CP01-DevSecOps (Grupo 2, 2026).

Código-fonte 3 – Automação do gate, DAST, SAST e encerramento

```
# Gate completo (JSON, SARIF, SBOM e resumo)
python3 scripts/run_scan.py vulneravel --offline      # esperado: 1 (FAIL)
python3 scripts/run_scan.py corrigido --offline      # esperado: 0 (PASS)
docker compose run --rm verify
python3 -m unittest scripts.test_gate -v

# DAST com Nuclei (somente alvos locais)
docker compose pull nuclei
docker compose --profile demo up -d --build --wait app-vulneravel app-corrigido
python3 scripts/run_dast.py

# SAST com SonarQube (painel em http://127.0.0.1:19000)
docker compose pull sonarqube sonar-scanner
docker compose --profile sonar up -d sonarqube
# Aguarde a mensagem de prontidão nos logs:
docker compose logs --tail=40 sonarqube
python3 scripts/setup_sonar.py
# Se ainda estiver iniciando, aguarde e repita setup_sonar.py.
python3 scripts/run_sonar.py

# Encerrar apenas este projeto, preservando volumes
docker compose --profile demo --profile sonar down
```

Fonte: Repositório CP01-DevSecOps (Grupo 2, 2026).

Código-fonte 4 – Workflow completo .github/workflows/security.yml

```

name: Security gate

on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:
  inputs:
    variant:
      description: Cenário didático (vulneravel deve falhar; corrigido deve
passar)
      required: true
      default: corrigido
      type: choice
      options: [vulneravel, corrigido]

permissions:
  contents: read

jobs:
  security:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    env:
      LAB_VARIANT: ${ inputs.variant || 'corrigido' }
    steps:
      - name: Checkout
        uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262 # v4
      - name: Validar Compose
        run: docker compose config --quiet
      - name: Verificar que erros não produzem falso verde
        run: python3 scripts/test_gate.py
      - name: Preparar scanners
        run: docker compose pull trivy kics
      - name: Executar SCA e IaC com gate HIGH/CRITICAL
        run: python3 scripts/run_scan.py "$LAB_VARIANT"
      - name: Preservar relatórios mesmo se o gate bloquear
        if: always()
        uses: actions/upload-artifact@eal65f8d65b6e75b540449e92b4886f43607fa02 # v4
        with:
          name: security-${ env.LAB_VARIANT }-${ github.run_id }
          path: |
            reports/trivy/${ env.LAB_VARIANT }/
            reports/kics/${ env.LAB_VARIANT }/
            reports/gate/${ env.LAB_VARIANT }/
          if-no-files-found: error
          retention-days: 30

```

Fonte: .github/workflows/security.yml no commit analisado (Grupo 2, 2026).

APÊNDICE C – TRECHOS ILUSTRATIVOS DE CONFIGURAÇÃO

Os exemplos a seguir apoiam a seção 4. Os digests abreviados e as omissões estão identificados; executar sempre os arquivos completos do commit analisado.

Código-fonte 5 – Trecho do docker-compose.yml: endurecimento dos alvos e serviço Trivy

```
x-app: &app
  read_only: true
  tmpfs:
    - /tmp:size=16m,noexec,nosuid
  cap_drop: [ALL]
  security_opt: [no-new-privileges:true]
  pids_limit: 64
  mem_limit: 256m
  cpus: 0.5
  networks: [lab]

services:
  trivy:
    image: aquasec/trivy:0.74.0@sha256:62b1e65e8869bc4b...87187c1969
    profiles: [tools]
    volumes:
      - ./fixtures:/workspace:ro
      - ./reports:/reports
      - trivy-cache:/root/.cache/trivy

networks:
  lab:
    internal: true
  sonar:
    driver: bridge
```

Fonte: docker-compose.yml no commit analisado (Grupo 2, 2026). Trecho ilustrativo com digest abreviado; não copiar como arquivo executável.

Código-fonte 6 – app/Dockerfile

```
ARG PYTHON_IMAGE=python:3.12-slim@sha256:2f17fc04...ebbe06a9
FROM ${PYTHON_IMAGE}
ARG VARIANT=corrigido
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY fixtures/${VARIANT}/requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r /app/requirements.txt \
    && groupadd --gid 10001 lab \
    && useradd --uid 10001 --gid 10001 --no-create-home lab
COPY app/server.py /app/server.py
USER 10001:10001
EXPOSE 8080
HEALTHCHECK --interval=5s --timeout=3s --start-period=10s --retries=10 \
  CMD python -c "import urllib.request;
  urllib.request.urlopen('http://127.0.0.1:8080/health', timeout=2)"
CMD ["python", "server.py"]
```

Fonte: app/Dockerfile no commit analisado (Grupo 2, 2026). Trecho ilustrativo com digest abreviado; usar o arquivo completo.

Código-fonte 7 – Trecho do securityContext no manifesto corrigido

```
containers:
  - name: app
    image: cp01-devsecops/app:lab-v1
    securityContext:
      privileged: false           # vulnerável: true
      allowPrivilegeEscalation: false # vulnerável: true
      readOnlyRootFilesystem: true   # vulnerável: false
      capabilities:
        drop: ["ALL"]
    resources:
      limits:
        cpu: 500m
        memory: 128Mi
```

Fonte: Adaptado de fixtures/corrigido/deployment.yaml (Grupo 2, 2026); comentários acrescentados.

Código-fonte 8 – nuclei/debug-exposure.yaml

```
id: cp01-debug-exposure
info:
  name: CP01 - Local lab internal configuration exposure
  severity: medium
  classification:
    cwe-id: CWE-497
http:
  - method: GET
    path:
      - "{{BaseURL}}/debug"
    redirects: false
    matchers-condition: and
    matchers:
      - type: status
        status: [200]
      - type: word
        part: body
        condition: and
        words:
          - '"lab_marker": "cp01-debug-exposure"'
          - '"internal_config"'
```

Fonte: nuclei/debug-exposure.yaml no commit analisado (Grupo 2, 2026). Trecho ilustrativo; o arquivo executável inclui author e demais metadados.